PROJET AWD • VALIDATION 1

Plateforme de constitution automatique de groupes

SKILLOOM

Présentation du sujet, de la modélisation microservices et de l’architecture proposée

THE SKILLWEAVERS • Année 2026–2027

1. Contexte et problématique

Le constat

La constitution manuelle des groupes peut être longue.
Les étudiants ont des niveaux, compétences, intérêts et styles d’apprentissage différents.

Le défi pédagogique

Comment former des groupes diversifiés et équilibrés tout en laissant à l’enseignant le contrôle de la répartition ?

Notre problématique

Comment automatiser la constitution de groupes d’apprentissage collaboratif à partir des profils étudiants, afin de favoriser la diversité, l’équilibre et la collaboration ?

La plateforme aide à la décision : l’enseignant conserve la validation finale.

2. Solution et objectifs

1 • Profils étudiants

Les étudiants renseignent leurs compétences, intérêts et styles d’apprentissage. Les informations académiques sont gérées par un acteur autorisé.

2 • Analyse intelligente

Le moteur ML analyse les profils et applique une méthode de clustering pour proposer une répartition hétérogène.

3 • Validation enseignante

L’enseignant consulte les propositions, ajuste la composition si nécessaire et valide les groupes.

Objectifs

Automatiser • équilibrer • gagner du temps • encourager la collaboration • suivre les activités

3. Acteurs et principales fonctionnalités

Étudiant

- Créer et mettre à jour son profil.
- Consulter son groupe.
- Voir les réunions planifiées.
- Consulter ses points et badges.

Enseignant

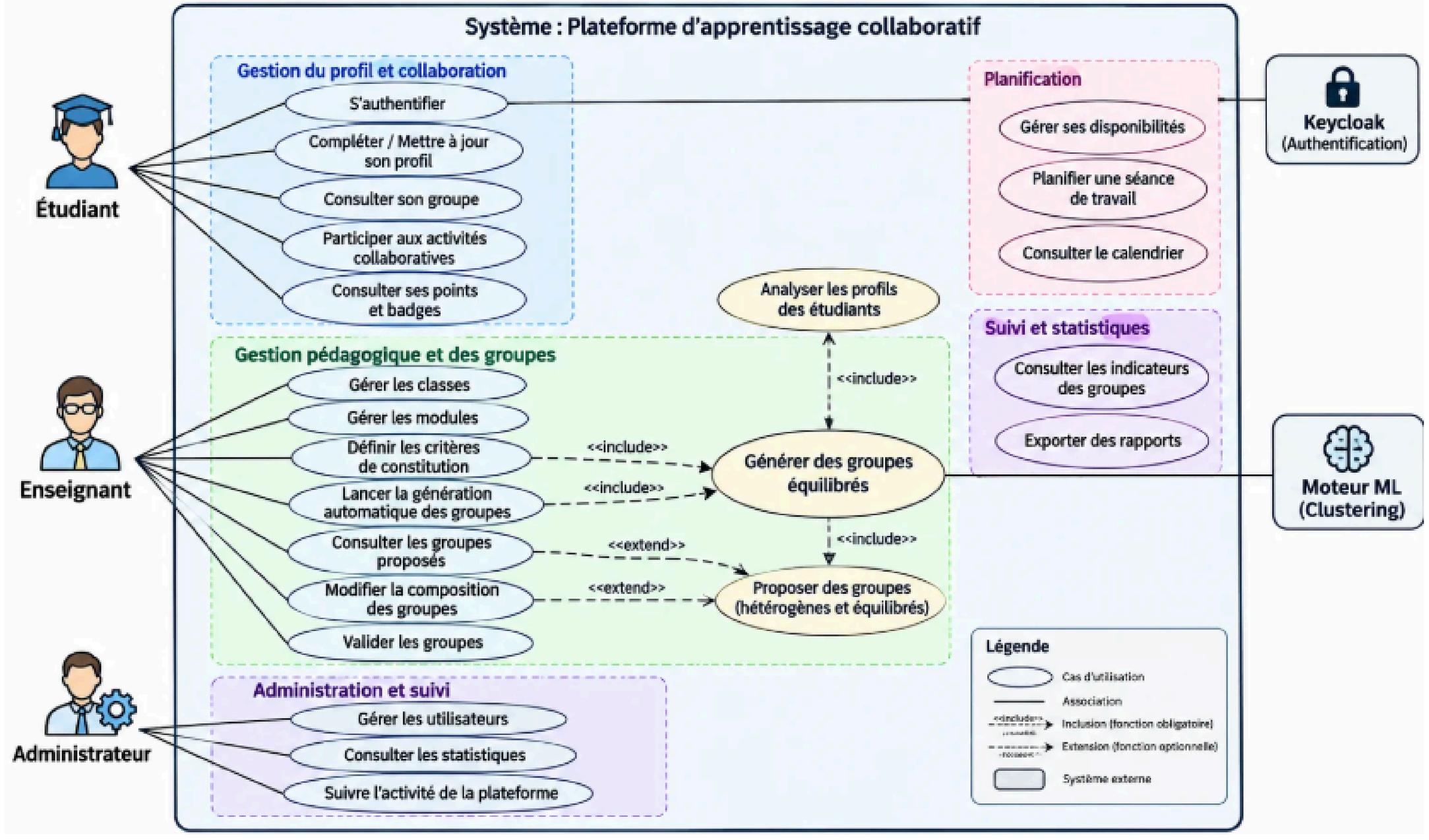
- Gérer les cours et les modules.
- Lancer la génération des groupes.
- Examiner les groupes proposés.
- Modifier et valider les groupes.
- Planifier les activités.

Administrateur

- Superviser la plateforme.
- Gérer les accès selon les rôles.
- Consulter les indicateurs globaux.

Diagramme de cas d'utilisation

Plateforme de constitution automatique de groupes d'apprentissage collaboratif



4. Scénario général de fonctionnement

1	**Profil**	Les étudiants renseignent leurs profils.
2	**Sélection**	L’enseignant choisit une classe et un module.
3	**Demande**	L’enseignant lance la génération des groupes.
4	**Collaboration**	Les étudiants consultent leurs groupes et collaborent.
5	**Proposition**	Le moteur retourne une proposition de groupes.
6	**Validation**	L’enseignant ajuste puis valide les groupes .
7	**Analyse ML**	Le moteur ML analyse les profils des étudiants.

5. Découpage proposé en microservices

Auth Service

Identité et accès applicatifs ; périmètre à clarifier avec Keycloak.

User / Profiling

Profils étudiants : compétences, intérêts, styles d’apprentissage.

Course / Group

Classes, modules, demandes, groupes, affectations et validation.

Scheduling Service

Disponibilités, réunions et calendriers.

Gamification Engine

Points, badges et règles de récompense.

Analytics Service

Indicateurs et statistiques agrégées.

ML Clustering Engine

Analyse des profils et proposition de groupes.

Découpage provisoire par capacités métier ; responsabilités à valider collectivement.

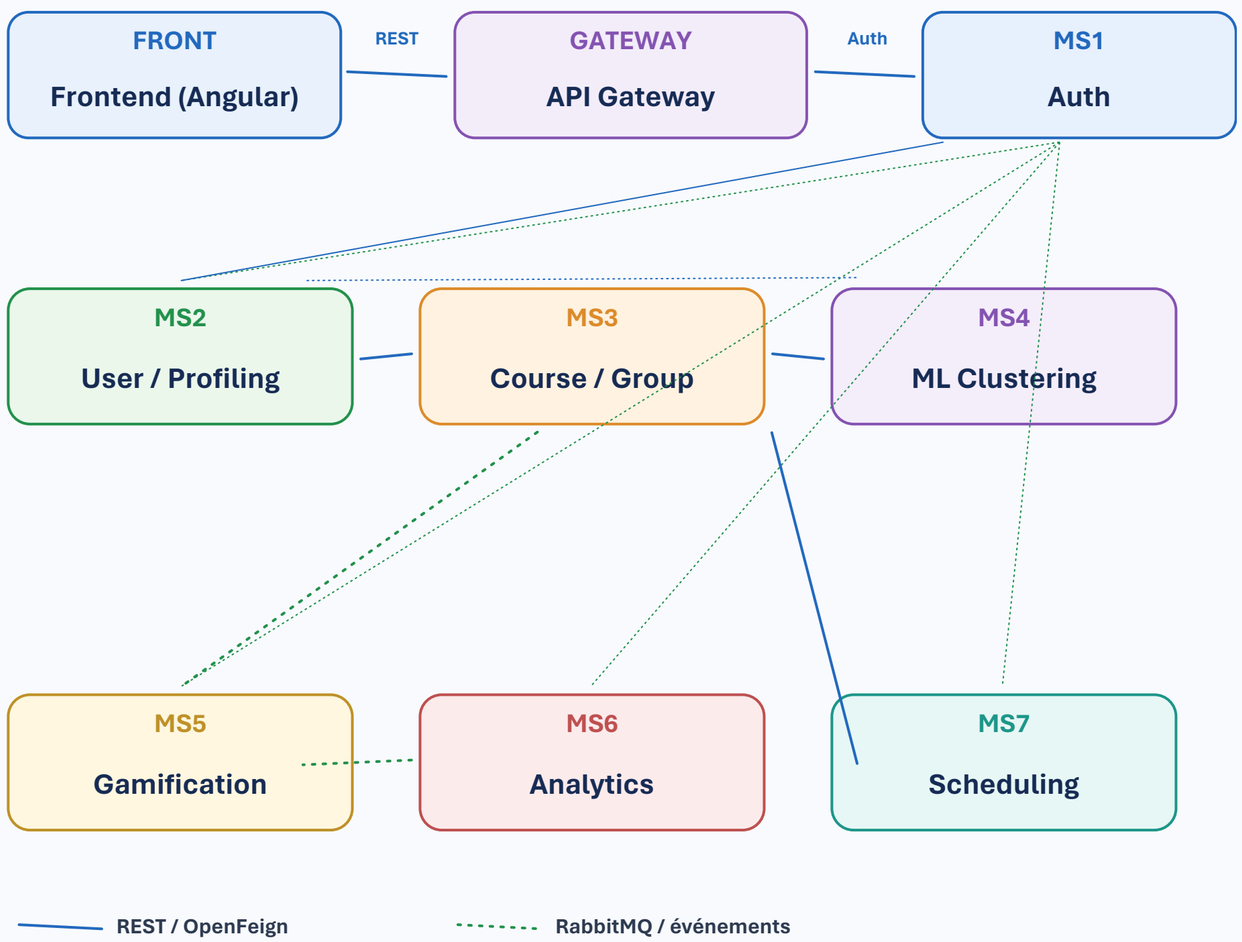
6. Données et propriété par service

Service	Données principales possédées	Base envisagée
Auth	Identité / droits	PostgreSQL
User / Profiling	Profil, compétences, intérêts, styles, données académiques autorisées	MySQL
Course / Group	Classes, modules, groupes, membres, demandes et versions	MySQL
Scheduling	Disponibilités, réunions et calendrier	MySQL
Gamification	Points, badges, récompenses	MySQL
Analytics	Indicateurs et agrégats	PostgreSQL
ML Clustering	Paramètres, exécutions et résultats du clustering	MongoDB

PostgreSQL (MS1, MS6) : adapté aux données structurées et aux opérations nécessitant une bonne cohérence.
MySQL (MS2, MS3, MS5, MS4) : adapté aux données relationnelles et aux associations entre entités.
MongoDB (MS7) : adapté au stockage des résultats de clustering et des documents contenant les clusters.

7. Interactions entre les services

Vue globale des communications



Chaque service conserve sa base de données ; aucun accès direct entre bases.

Liste complète des échanges

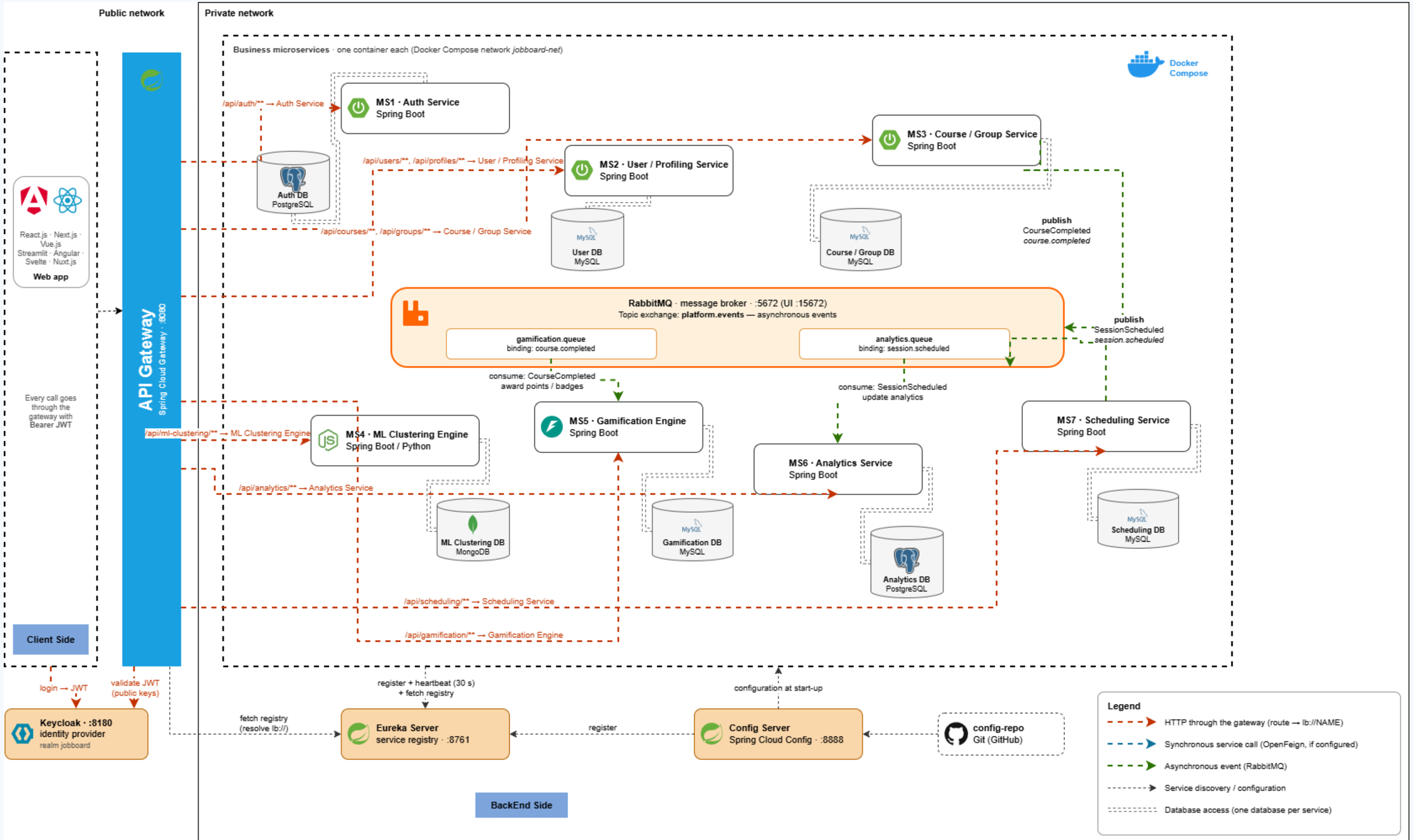
SYNCHRONES — REST / OpenFeign

MS2 → MS1	GET /api/auth/users/{id} — vérification du compte
MS3 → MS2	GET /api/profiles/{id} — profils tuteur / étudiants
MS4 → MS2	GET /api/profiles/{id} — données de clustering
MS3 → MS4	POST /api/ml-clustering/runs — demande ML
MS7 → MS3	GET /api/groups/{id} — consultation du groupe

ASYNCHRONES — RABBITMQ

MS1 → MS2 / MS5 / MS6 / MS7	Événements compte : registered, role-changed, deleted
MS3 → MS5	CourseCompleted — fin d'inscription au cours
MS7 → MS6	SessionScheduled — nouvelle séance planifiée

8. Architecture technique globale proposée



9. Technologies et principes d’architecture

Backend et communication

Spring Boot
Spring Cloud Gateway
Eureka
REST / OpenFeign

Sécurité et configuration

Keycloak
Config Server
Rôles et tokens

Données et déploiement

MySQL
MongoDB / PostgreSQL
Docker / Docker Compose

Principes retenus

- Un microservice correspond à une capacité métier délimitée.
- Chaque service possède et gère sa propre base de données.
- Les échanges passent par des contrats explicites.
- Un service ne doit pas accéder directement à la base d’un autre service.

Les choix finaux de bases et la technologie frontend restent à confirmer par l’équipe.

10. Organisation de l’équipe et du dépôt Git

Structure du dépôt

Backend/ : un projet par microservice
Frontend/ : application cliente
Documentation/Presentation
Documentation/Diagrams
README.md

Règles de collaboration

Dépôt Git public
Enseignant ajouté comme collaborateur
Commits personnels réguliers
Suivi des tâches visible

Répartition proposée — compléter avec les noms réels

Auth Service	youseef Rbai
User / Profiling	Rayen ben Yahia
Course / Group	Meriem Ghrib
Scheduling	Nedra Ben Charrada
Gamification	Yessmine Snoussi
Analytics	Sarra Souidi
ML Clustering Engine	Toute l’equipe

À confirmer : répartition exacte, responsable du service avancé et outil de gestion du projet.

11. Planning prévisionnel et risques

Planning indicatif

1. Valider le sujet et le périmètre
2. Concevoir les microservices et les données
3. Définir les contrats API et interactions
4. Créer les projets et organiser Git
5. Développer progressivement
6. Intégrer, tester et documenter

Risques et mesures

- Responsabilités qui se chevauchent → préciser les limites métier.
- Données incohérentes → définir la propriété des données.
- Service indisponible → prévoir erreurs et reprises.
- Complexité ML → définir entrées, sorties et critères.
- Retard d'intégration → commits fréquents et suivi des tâches.

Pour la validation intermédiaire : priorité à la conception, à l'organisation Git et à la répartition du travail ; la logique métier n'est pas attendue.

Auth Service : conception

Responsable : youseef Rbai

Responsabilité métier

Authentification a l application en suivant les normes de securite

Base : Postgresql(auth)

PostgreSQL convient aux données d'identité et d'audit : contraintes d'unicité, intégrité transactionnelle et requêtes d'historique.

API REST : endpoints représentatifs

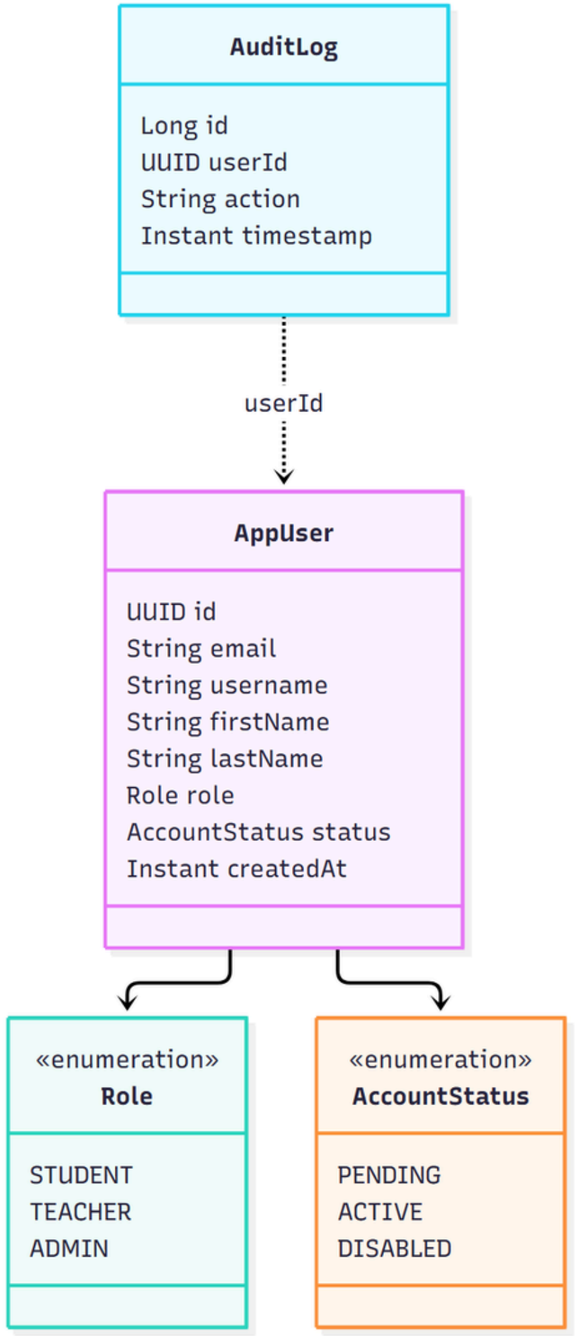
POST	/api/auth/register	Créer un compte	201 · 400 · 409 ·
POST	/api/auth/login	Consulter un profil	200 · 401 · 404
PUT	/api/auth/me/password	changer un mot de passe	204 400 401
GET	/api/auth/users	Lister / rechercher (accès interne)	200 · 401 · 403

payload: log in

{ "email": "amira@example.com",
"password": "S3cure!pass" }

200
{
"accessToken": "eyJhbGciOi...",
"refreshToken": "eyJhbGciOi...",
"tokenType": "Bearer",
"expiresIn": 300
}

Diagramme de classe : Auth



Interactions avec les autres microservices

MS1 — Auth Service • **Responsable : Youssef**



- **Synchrone : appel REST et réponse immédiate.**
- **Asynchrone : événement publié via RabbitMQ, traité par MS2, MS5 et MS6.**

User / Profiling Service : conception

Responsable : Rayen ben yahia

Responsabilité métier

Centraliser le profil pédagogique de chaque utilisateur : niveau académique, style d'apprentissage, disponibilités, intérêts et compétences. MS2 fournit ces données à MS3 pour la constitution des groupes et à MS4 pour le clustering.

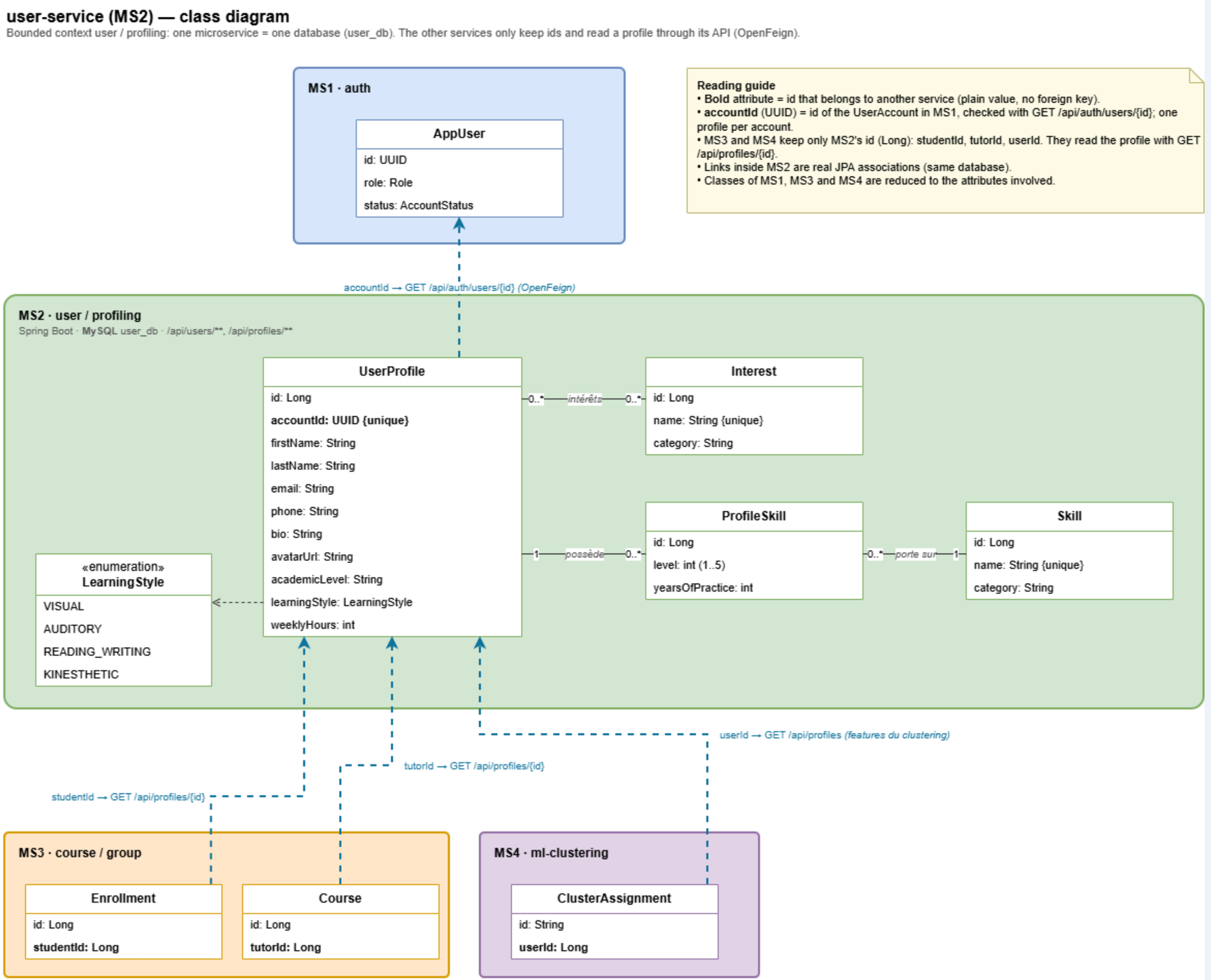
Base : MySQL (user_db)

- UserProfile : accountId (UUID unique), identité et attributs pédagogiques
- Interest : nom unique, catégorie
- Skill : nom unique, catégorie
- ProfileSkill : niveau (1–5), années de pratique
- Base privée : seul MS2 lit / écrit user_db

API REST : endpoints représentatifs

POST	`/api/profiles`	Créer le profil d'un compte	**201 · 400 · 401 · 409 · 503**
GET	`/api/profiles/{id}`	Consulter un profil	**200 · 401 · 404**
PUT	`/api/profiles/{id}`	Mettre à jour un profil	**200 · 400 · 401 · 404**
GET	`/api/profiles`	Lister / rechercher (accès interne)	**200 · 401 · 403**

Modèle de données



Interactions avec les autres microservices

Synchrone : REST et OpenFeign

Appels directs — données lues chez leur propriétaire

Exemples : création et consultation d'un profil

• MS2 → MS1 : GET /api/auth/users/{id} pour vérifier accountId avant création (OpenFeign).
• MS3 → MS2 : GET /api/profiles/{id} pour lire le profil étudiant ou tuteur.
• MS4 → MS2 : GET /api/profiles/{id} pour récupérer les attributs utilisés par le clustering.

Pourquoi : MS2 reste propriétaire des données de profil ; les autres services ne stockent que les IDs.

Principe : pas de copie de user_db dans MS3 ou MS4.

Asynchrone : pas d'événement MS2 défini

MS2 n'utilise pas RabbitMQ dans le flux décrit

Dans l'architecture globale :

• RabbitMQ transporte notamment CourseCompleted et SessionScheduled entre d'autres services.
• Aucun événement publié ou consommé par MS2 n'est représenté dans le diagramme global.
• Les échanges MS1 / MS2 / MS3 / MS4 liés au profil sont donc synchrones via HTTP / OpenFeign.

Évolution possible : publier ProfileUpdated si un besoin métier de synchronisation apparaît.

Ne pas présenter un événement comme déjà implémenté.

Gestion des pannes :
Si un service tombe : MS1 indisponible lors de la création → MS2 renvoie une erreur contrôlée (503) et ne crée pas de profil orphelin. MS2 indisponible → MS3 / MS4 reçoivent une erreur temporaire (503) et peuvent réessayer avec timeout / circuit breaker. Un profil déjà enregistré reste en base.

Course / Group Service : conception

MS3 — Course / Group Service • Responsable : Maryem

Spring Boot • Java 17 • MySQL (course_db) • Port 8083 • API : /api/courses et /api/groups

Responsabilités métier

• Gérer les cours / modules et les inscriptions des étudiants.
• Créer et gérer les groupes d'étude et leurs membres.
• Permettre à l'enseignant de modifier et valider la composition des groupes.
• Demander au moteur ML la génération des groupes et suivre le résultat.
• Publier CourseCompleted lorsqu'une inscription est terminée.

Base de données : MySQL

• Données fortement relationnelles : cours, groupes, membres et inscriptions.
• Clés étrangères et contraintes pour préserver la cohérence des associations.

Contrats d'API REST : principaux endpoints

Méthod	Endpoint	Fonction	Réponses HTTP
POST	/api/courses	Créer un cours	201 · 400 · 401 · 403
GET	/api/courses	Lister les cours	200 · 401
GET	/api/courses/{id}	Consulter un cours	200 · 401 · 404
POST	/api/courses/{id}/enrollments	Inscrire un étudiant	201 · 400 · 404 · 409 · 503
POST	/api/courses/{id}/group-generation-requests	Demander la génération ML	202 · 400 · 404 · 503
DELETE	/api/courses/{id}	Supprimer un cours	204 · 403 · 404 · 409

Interactions avec les autres microservices

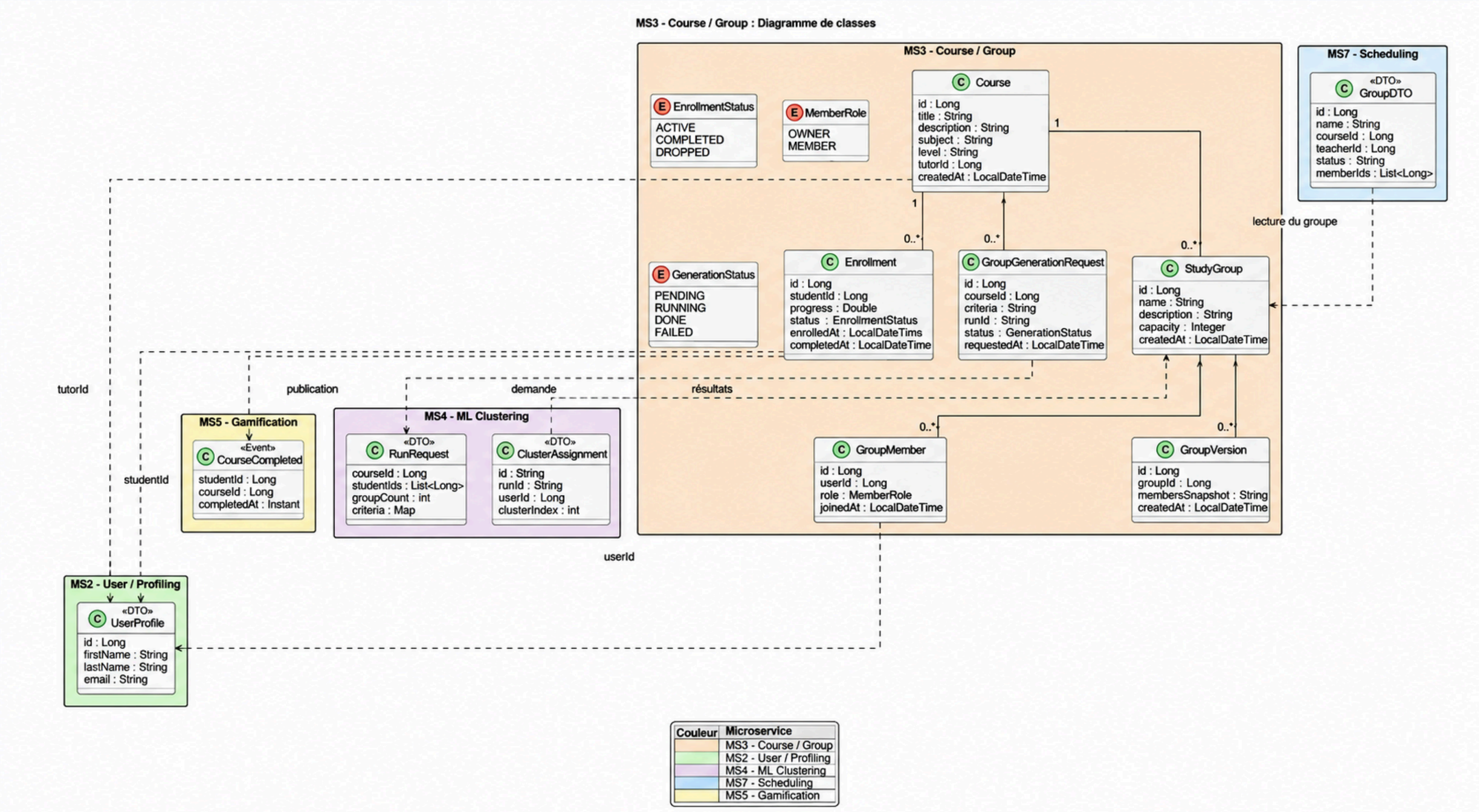
MS3 — Course / Group Service • Responsable : Maryem



- **Synchrone : appel REST et réponse immédiate.**
- **Asynchrone : publication d'un événement traité par RabbitMQ.**

Diagramme de classes : Course / Group :

MS3 — Course / Group Service • Responsable : Maryem



Scheduling Service : conception

Responsable : Nedra Ben Charrada

Responsabilité métier

Planifier les sessions de travail d'un groupe (KICKOFF, WORK_SESSION, REVIEW, DEFENSE), gérer les disponibilités, les salles, les deadlines et les rappels, puis publier les événements de planning.

Base : MySQL (scheduling_db)

- Données relationnelles : session, participants, rappels
- Transactions ACID : pas de chevauchement de créneaux
- Base privée : aucun autre service ne la lit

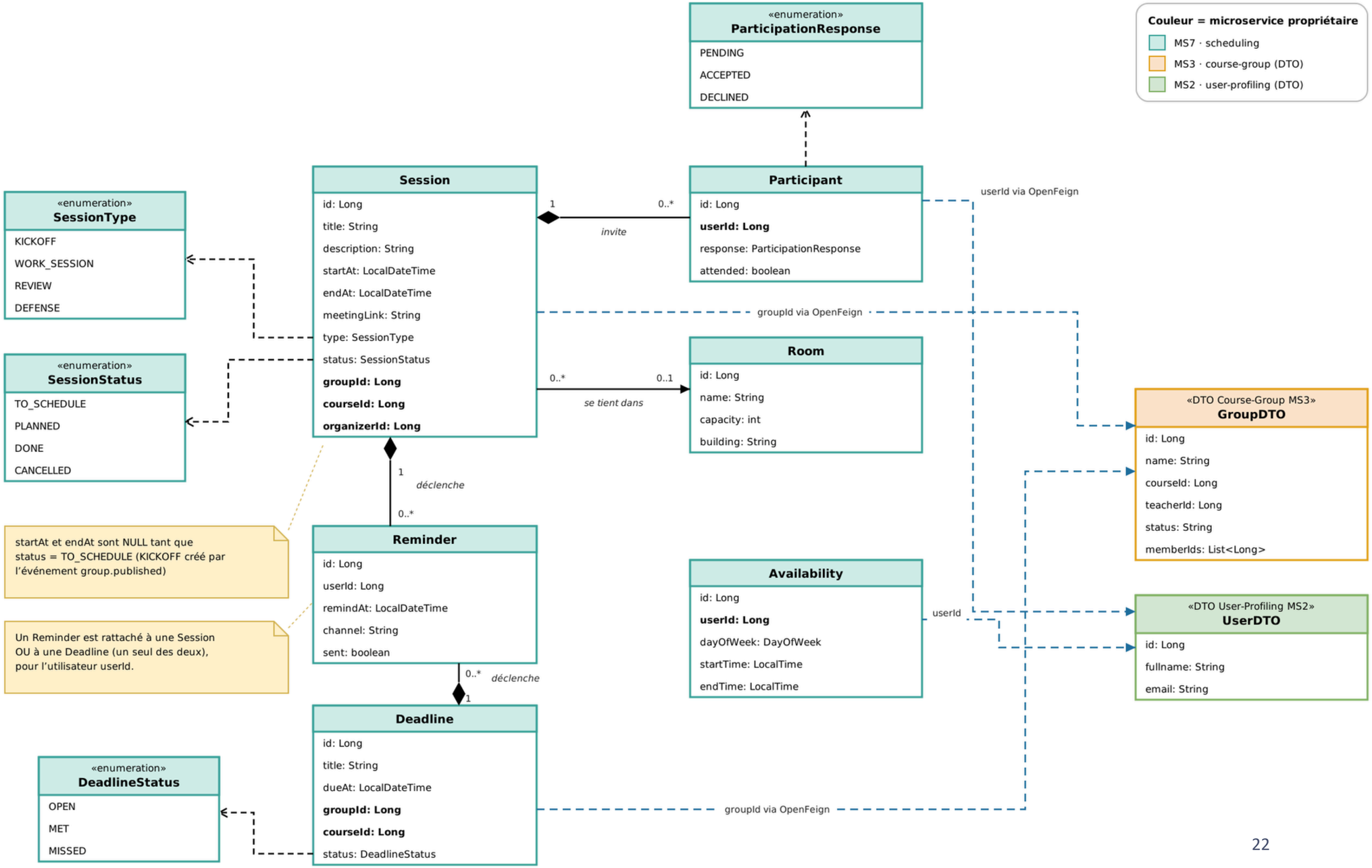
API REST — endpoints représentatifs

POST	`/api/sessions`	Planifier une réunion	**201 · 400 · 404 · 409 · 503**
PUT	`/api/sessions/{id}/attendance`	Enregistrer la présence	**200 · 403 · 404**
POST	`/api/availabilities`	Déclarer une disponibilité	**201 · 400**
GET	`/api/sessions?groupId=`	Lister les réunions d'un groupe	**200 · 400**

Modèle de données

Scheduling (MS7) — diagramme de classes

Modèle du microservice Scheduling (scheduling_db, MySQL). Les DTO viennent des autres services via OpenFeign ; seuls les ids sont stockés.



Interactions avec les autres microservices

Synchrone : OpenFeign

Exemple : planifier une réunion

- **Nuxt.js → Gateway → MS7 :** POST /api/sessions
- **MS7 → MS3 :** GET /api/groups/3, reçoit courseId et memberIds
- **MS7 → MS2 :** GET /api/users?ids=4,7,9, reçoit fullname et email
- **MS7 → client :** 201 Created

Pourquoi : la réponse est nécessaire pour valider avant d'écrire.

Asynchrone : RabbitMQ

Exemple : vie d'un groupe

- **MS3 → MS7 :** group.published, MS7 crée un KICKOFF en TO_SCHEDULE
- **MS7 → MS6 :** session.scheduled, publié après COMMIT, mise à jour des statistiques
- **MS7 → MS5 et MS6 :** attendance.recorded, badge de régularité et sessionsAttended

Pourquoi : le client n'attend pas et les services restent découplés.

Si un service tombe : MS3 ou MS2 indisponible : circuit breaker, 503, rien n'est écrit. RabbitMQ ou consommateur en échec : événement rejoué (outbox), puis scheduling.dlq après 3 échecs.

Microservice — Gamification Engine

Responsable : Yessmine Snoussi

Responsabilité métier

Gérer les mécanismes de motivation et de récompense des étudiants à travers l'attribution de points, de badges et de niveaux selon leurs contributions aux activités collaboratives.

Base : MySQL (gamification_db)

- StudentGamification (id, studentId, totalPoints, currentLevel, avatarId)
- PointTransaction (id, studentId, points, reason, eventId, date)
- Badge (id, name, description, condition)
- StudentBadge (id, studentId, badgeId, earnedAt)
- Level (id, name, requiredPoints)
- Avatar (id, name, imageUrl, requiredPoints)

API REST — endpoints représentatifs

POST	`/api/gamification/badges`	Créer un badge	**(201, 400, 403, 409)**
PUT	`api/gamification/students/{id}/avatar`	Modifier l'avatar	**(200 / 400 / 404)**
GET	`/api/gamification/badges`	Consulter les badges disponibles	**200**
GET	`/api/gamification/students/{id}/points`	Consulter les points	**200 / 404**

Modèle de données

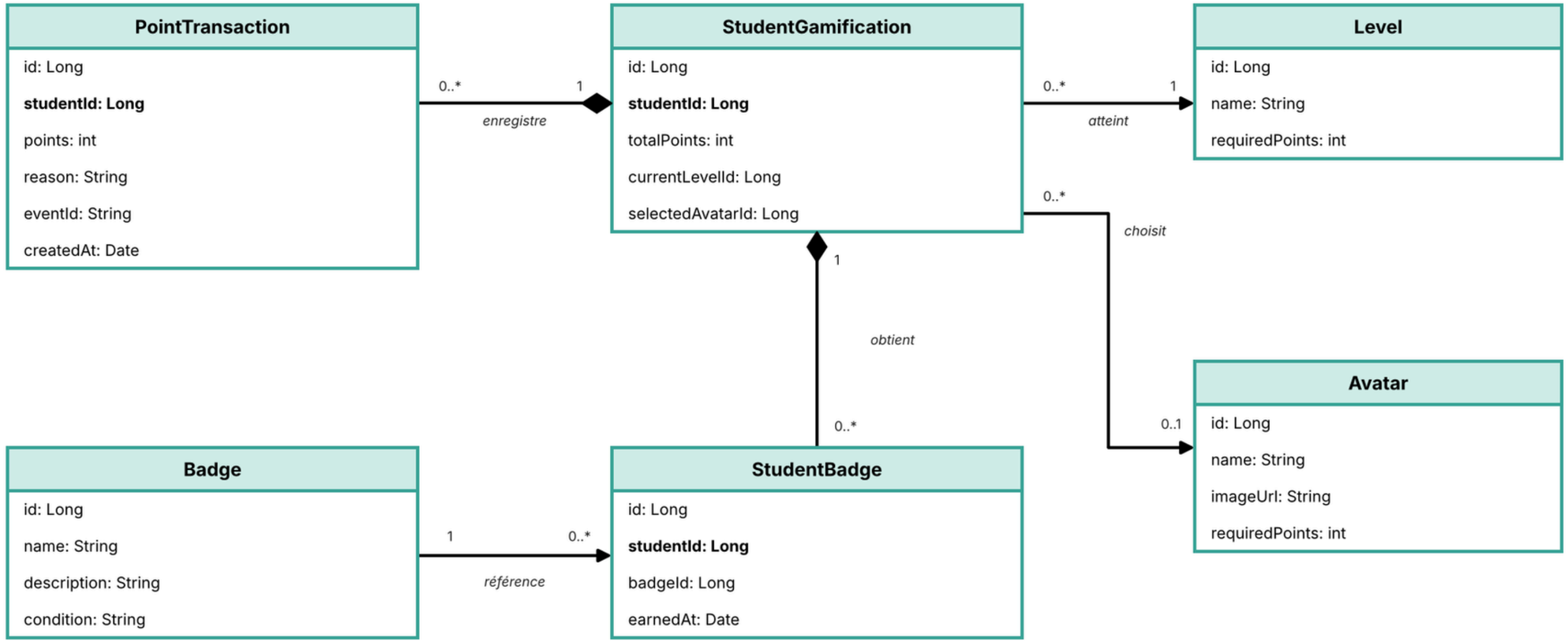
Gamification — diagramme de classes

Modèle du microservice Gamification. studentId référence l'étudiant du User Service ; seules les données de gamification sont stockées.

Couleur = microservice propriétaire

Gamification · classes

Aa id en gras = référence logique vers un autre service



studentId référence l'étudiant dans le User Service.
Les données restent dans la base du Gamification Service.

Interactions avec les autres microservices

Synchrone :

Exemple : consulter le profil gamification d'un étudiant

- **Angular→ Gateway → MS5 :** GET /api/gamification/students/42
- **MS5 → MS2 :** GET /api/profiles/42 (OpenFeign), reçoit les informations de l'étudiant et vérifie son existence.
- **MS5 → client :** 200 OK avec le profil gamification.

la réponse de MS2 permet de vérifier l'existence de l'étudiant avant de retourner son profil gamification.

Asynchrone :

Exemple : attribution automatique des récompenses

- **MS3 → MS5 :** CourseCompleted (RabbitMQ), MS5 reçoit studentId, eventId et les informations de l'activité.
- **MS5 :** vérifie l'unicité de eventId dans PointTransaction, attribue les points, met à jour le niveau et vérifie les conditions d'obtention des badges.
- **MS5 → MS6 :** RewardEarned (RabbitMQ), publié après COMMIT, transmet les informations nécessaires à la mise à jour des statistiques.

MS3 n'attend pas le calcul des récompenses. La communication asynchrone permet de découpler les services et de ne pas ralentir les opérations pédagogiques.

Si un service tombe :

- MS2 indisponible : Circuit Breaker et retour 503 Service Unavailable.
- RabbitMQ / MS5 en échec : Retry, puis transfert vers gamification.dlq après 3 tentatives.
- Événement en double : Vérification de eventId pour éviter les doubles récompenses.
- MS6 indisponible : Événement conservé dans RabbitMQ jusqu'à sa reprise.

Microservice — Analytics Service

Responsable : Sarra Souidi

Responsabilité métier

Collecter, centraliser et analyser les statistiques des groupes et de l'utilisation de la plateforme afin de fournir des indicateurs et des tableaux de bord.

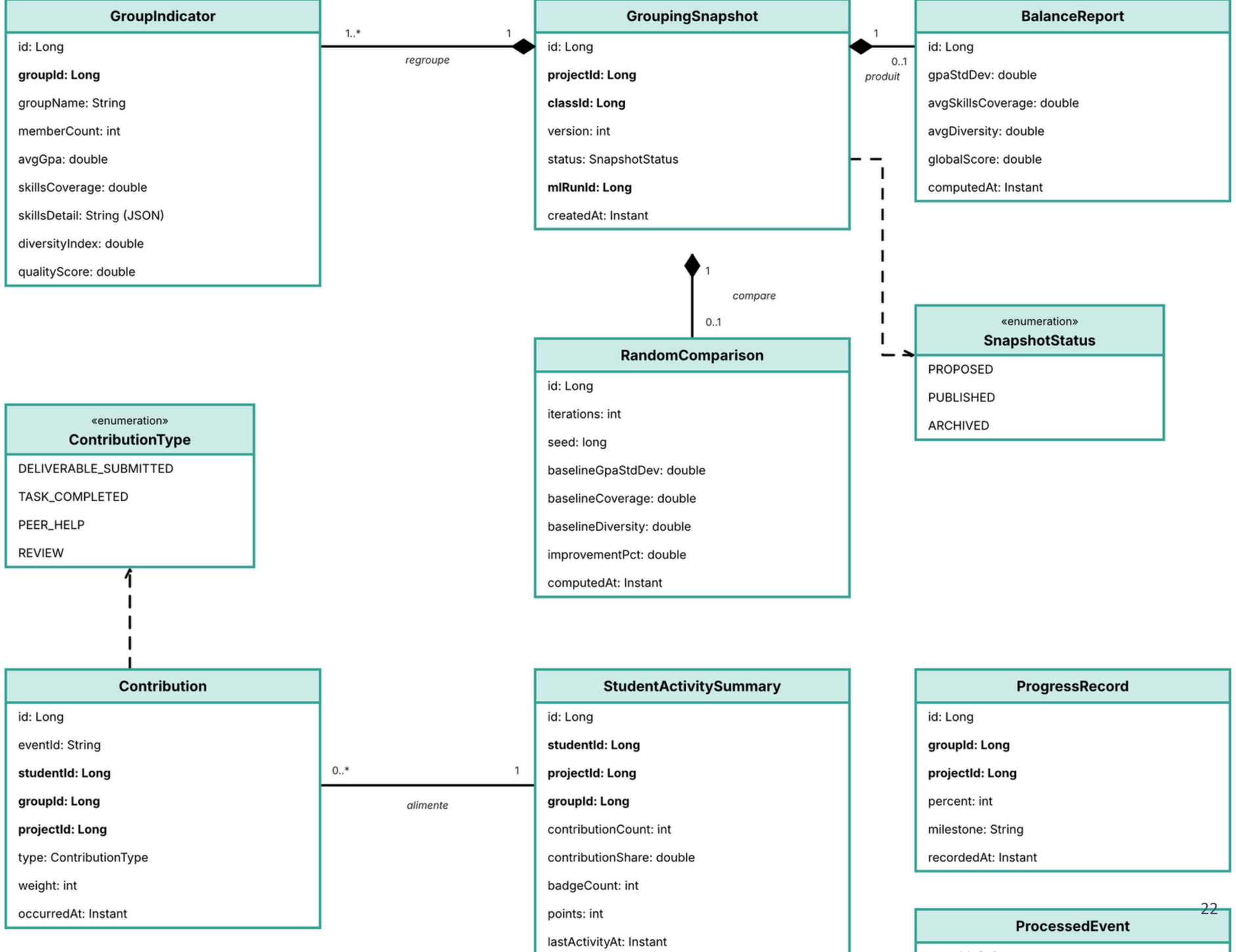
Base de données : PostgreSQL (analytics_db)

- Données relationnelles : statistiques des groupes, des étudiants et des activités.
- Transactions ACID : garantir la cohérence et la fiabilité des statistiques.
- Base privée : aucun autre microservice ne lit directement cette base.

API REST — endpoints représentatifs

POST	`/api/analytics/events`	Recevoir un événement à analyser	**201 · 400 · 404 · 409 · 503**
PUT	`/api/analytics/groups/{groupId}/statistics`	Mettre à jour les statistiques d'un groupe	**200 · 403 · 404**
POST	`/api/analytics/snapshots/{snapshotId}/comparison`	Lancer une comparaison avec une répartition aléatoire	**201 · 400**
GET	`/api/analytics/overview`	Consulter les statistiques générales	**200 · 400**

Modèle de données



Interactions avec les autres microservices

Synchrone : OpenFeign

Exemple : comparer une répartition à l'aléatoire

- **Svelte → Gateway → MS6 :** POST /api/analytics/snapshots/12/comparison
- **MS6 → MS2 :** GET /api/profiles?ids=, reçoit gpa et compétences
- **MS6 → MS3, MS4, MS5 :** groupes, qualité du run, stats du joueur
- **MS6 → client :** 202 Accepted

Pourquoi : le calcul ne peut pas continuer sans la donnée.

Asynchrone : RabbitMQ

Exemple : répartition proposée

- **MS3 → MS6 :** grouping.proposed, crée un snapshot PROPOSED, ses indicateurs et le rapport d'équilibre
- **MS3 → MS6 :** grouping.validated, le snapshot passe à PUBLISHED
- **MS3 et MS5 → MS6 :** contribution.recorded, points.updated, badge.awarded, mise à jour des statistiques d'activité

Pourquoi : le client n'attend pas et les services restent découplés.

Si un service tombe : MS2 indisponible : circuit breaker, 503, aucune valeur partielle écrite. MS3, MS4 ou MS5 indisponible : dernières données stockées, stale: true. Message en erreur : analytics.dlq après 3 échecs ; ProcessedEvent évite les doublons.

ML Clustering Engine : conception

Responsable : Toute l’équipe (microservice commun) · présenté par : [Nom Prénom]

Responsabilité métier

Prétraiter les profils, exécuter le clustering (k-means, k-prototypes), répartir les étudiants en groupes équilibrés et calculer des indicateurs de qualité. Le moteur propose, l’enseignant valide.

Base : MongoDB (clustering_db)

- Paramètres et centroïdes de structure variable
- Clusters et qualité embarqués dans le run
- Base privée : aucun autre service ne la lit

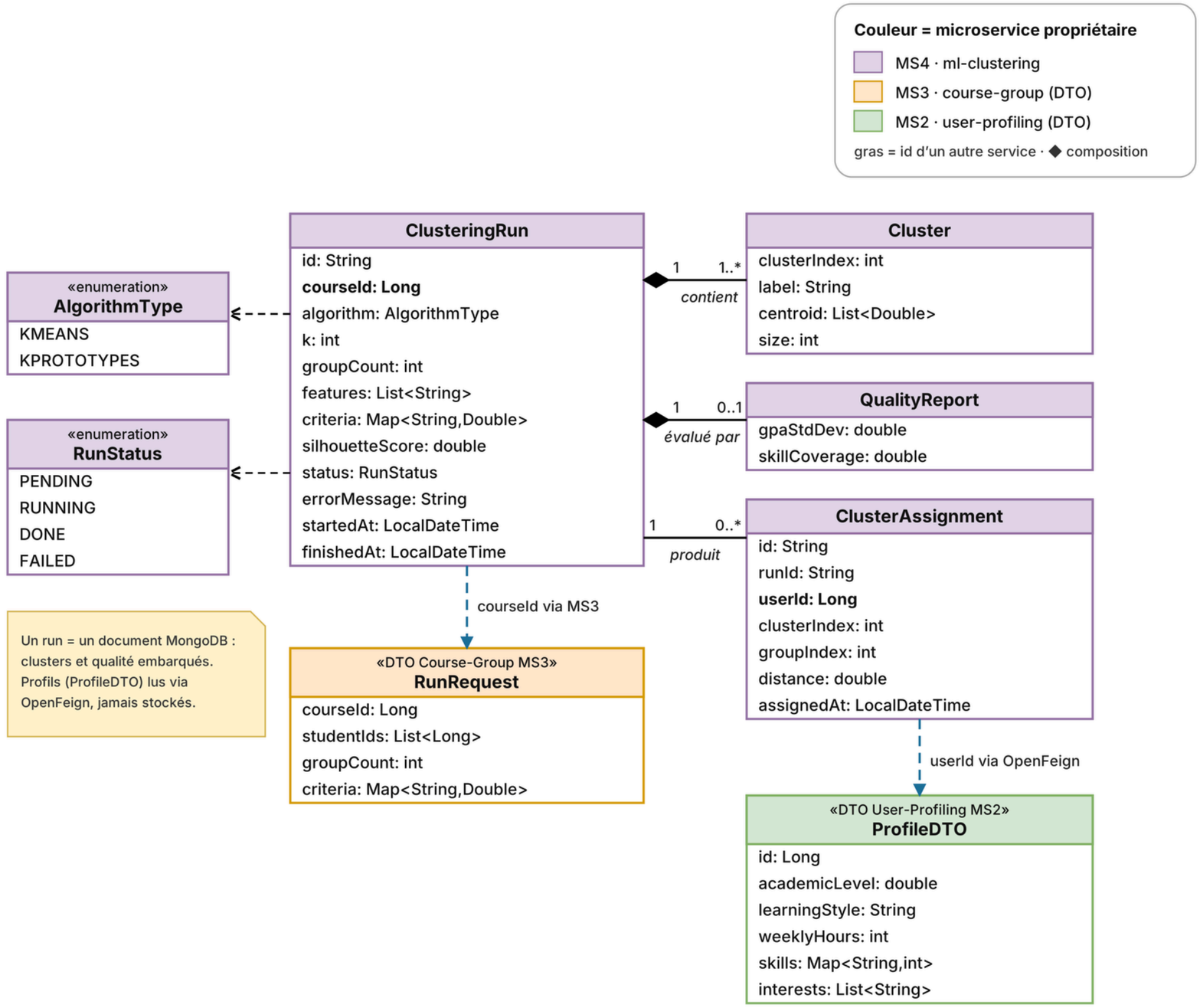
API REST : endpoints représentatifs

POST	`/api/ml-clustering/runs`	Lancer une exécution	**202 · 400 · 403 · 503**
GET	`/api/ml-clustering/runs/{runId}`	Statut d’une exécution	**200 · 404**
GET	`/api/ml-clustering/runs/{runId}/assignments`	Groupes proposés	**200 · 404 · 409**
GET	`/api/ml-clustering/runs/{runId}/quality`	Indicateurs de qualité	**200 · 404**

Modèle de données

ML Clustering (MS4) — diagramme de classes

Modèle du microservice ML Clustering Engine (clustering_db, MongoDB). Les DTO viennent des autres services ; seuls les ids sont stockés.



Interactions avec les autres microservices

Synchrone : REST et OpenFeign

Exemple : lancer une génération

- **MS3 → MS4 :** POST /api/ml-clustering/runs, reçoit 202 et le runId
- **MS4 → MS2 :** GET /api/profiles?ids=4,7,9,11 (OpenFeign), reçoit les features
- **MS3 → MS4 :** GET /api/ml-clustering/runs/{runId}/assignments, reçoit les groupes proposés

Pourquoi : MS4 lit les profils chez leur propriétaire, sans les copier.

Asynchrone : 202 + suivi du run

Exemple : exécution d'un run

- **MS4 :** répond 202, puis calcule en arrière-plan : PENDING, RUNNING, DONE
- **MS3 → MS4 :** GET /api/ml-clustering/runs/{runId} jusqu'à DONE ou FAILED
- **Évolution possible :** événement RabbitMQ clustering.completed (MS4 → MS3), sans interrogation

Pourquoi : le calcul est long, le client n'attend pas.

Si un service tombe : MS2 indisponible : le run passe à FAILED avec un message explicite et peut être relancé. MS4 indisponible : MS3 reçoit un 503 (circuit breaker). Un run DONE reste consultable.

CONCLUSION

Notre plateforme vise à aider les enseignants à constituer des groupes diversifiés grâce à l’analyse des profils étudiants, tout en conservant leur contrôle sur la validation finale.

Merci pour votre attention

À compléter : noms • bases finales • décision Auth / Keycloak • contrats API